

Zero to Hero Study Handbook: NL2SQL Agent

How to use this handbook:

1. Read modules in order. Each module builds on the previous one.
2. Keep the code open while reading. Every concept here maps to real files/functions in this repo.
3. After Module 5, use the checklist to validate end-to-end understanding.

Module 1: Foundations & Architecture

1.1 What this project does

`nl2sql-agent` is a local-first natural-language-to-SQL system. A user asks a plain-English question (CLI or Streamlit UI), the agent generates SQL, validates it for safety, runs it on SQLite, and returns a natural-language answer plus SQL/result context.

Primary use cases in this repo:

1. Local analytics assistant over a small SQLite database (`company.db` by default).
2. Safe text-to-SQL experimentation with policy controls (joins, subqueries, aggregates, CTE).
3. Multi-provider LLM runtime where Ollama is default, and OpenAI/Gemini/Anthropic are optional.

1.2 Core paradigms and patterns used

Definitions first, then where they appear in this codebase:

1. Modular architecture: Definition: split the system into small modules with one clear job each. In this repo: `config`, `db`, `security`, `llm`, `prompts`, `agent`, `ui`, `utils` under `src/nl2sql_agent/`.
2. Factory pattern: Definition: one function returns different implementations behind a shared interface. In this repo: `build_chat_model()` returns provider-specific chat models in `src/nl2sql_agent/llm/factory.py`.
3. Stateful workflow (graph/state-machine orchestration): Definition: execution is modeled as nodes and conditional transitions over shared state. In this repo: `NL2SQLAgent.get_workflow()` in `src/nl2sql_agent/agent/workflow.py` using LangGraph.
4. Policy-based security (allow-list): Definition: explicitly allow known-safe behavior instead of only blocking known-bad patterns. In this repo: `SQLPolicy + validate_sql()` in `src/nl2sql_agent/security/sql_validator.py`.
5. Typed data contracts: Definition: key data objects have explicit types and field names. In this repo: `AgentState` (`agent/state.py`) and `QueryResult` (`db/database.py`).

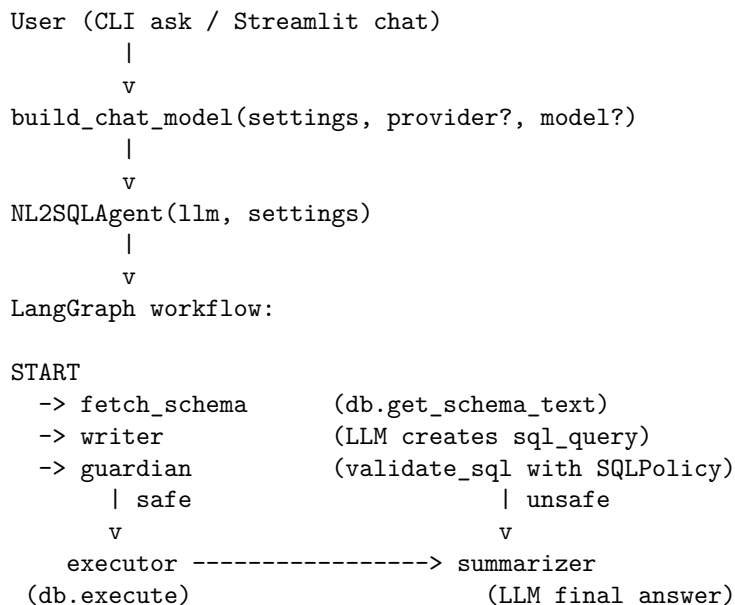
6. Settings singleton with environment binding: Definition: one cached settings object is reused across the app and can be reset in tests. In this repo: `Settings`, `get_settings()`, `reset_settings_cache()` in `config/settings.py`.

1.3 Architecture description

Key components and interactions:

1. Entrypoints:
 - CLI: `src/nl2sql_agent/cli.py` (`main()`, subcommands `ask/config/serve`)
 - UI: `src/nl2sql_agent/ui/streamlit_app.py` (`main()`)
2. Model construction:
 - `build_chat_model()` selects provider and returns a `BaseChatModel`.
3. Core agent workflow:
 - `NL2SQLAgent` builds a `LangGraph` with nodes: `fetch_schema` -> `writer` -> `guardian` -> `executor` -> `summarizer`
4. Data access:
 - `Database.ensure_schema()` creates/seeds tables.
 - `Database.execute()` runs SQL and returns `QueryResult`.
5. Security gate:
 - `validate_sql()` blocks non-SELECT, multi-statement, dangerous functions, and policy violations.
6. Prompting:
 - SQL-writing and summarization prompts in `prompts/templates.py`.

1.4 Main flow diagram (ASCII)



```

| ok
| error and retry_count < max_retries
+-----> writer (retry)

```

summarizer -> END

Module 2: Repository Map

Files a new contributor should learn first (in practical reading order):

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
pyproject.toml	Packaging, dependencies, scripts, lint/type/test config	[project.scripts]nl2sql-agent	requires-python, dependency pins, Ruff/Mypy/Pytest settings
src/nl2sql_agent/CLI.py	CLI entrypoint for ask/config/serve	main, build_parser, cmd_ask, cmd_config, cmd_serve, _build_agent	CLI args: --provider, --model, --api-key, --show-sql, --port, --host
src/nl2sql_agent/StreamlitApp.py	Streamlit interface and chat orchestration	main, _init_session_state	st.session_state.messages, st.session_state.ollama_base_url, refresh_trigger*
src/nl2sql_agent/UI/components.py	UI components rendering helpers	render_sidebar, render_chat_history, render_run_result, _default_models	Sidebar return keys: provider, model, api_key, ollama_base_url
src/nl2sql_agent/Agent/workflow.py	Agent workflow LangGraph workflow and routing	NL2SQLAgent, fetch_schema, write_sql, check_security, execute_sql, summarize_result, route_after_security, route_after_execute, run, stream	State keys: question, sql_query, error, retry_count, max_retries, final_answer
src/nl2sql_agent/Agent/state.py	Agent state contract	AgentState	Fields for schema, SQL, retries, raw rows, columns, row_count

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/nl2sql_agent/Security/sqlvalidation.py	Security, sql validation and allow-list enforcement	SQLPolicy, validate_sql, parse_sql, referenced_tables, SQLValidationError	DANGEROUS_FUNCTIONS, policy flags (allow_subqueries, etc.)
src/nl2sql_agent/SQL/database.py	Database copy tion/schema/query layer	Database, QueryResult, ensure_schema, get_schema_text, execute, setup_db, render_table	DDL_DEPARTMENTS, DDL_EMPLOYEES, timeout_seconds, max_rows
src/nl2sql_agent/DB/seed.py	DB/seed.py data	SEED_DEPARTMENTS, SEED_EMPLOYEES	Department/employee tuples loaded via INSERT OR IGNORE
src/nl2sql_agent/Provider/provider.py	Provider, specify model building and model listing	build_chat_model, list_models, fallback_models, _build_*, _list_*, LLMProviderError	Provider literals: ollama/openai/gemini/anthropic; fallback model tuples
src/nl2sql_agent/Config/settings.py	Config/settings.py config (Pydantic Settings)	Settings, get_settings, reset_settings_callback, env_var_for, env_var_value	env_prefix="NL2SQL_", .env, provider/db/llm/sql/log fields
src/nl2sql_agent/Prompts/templates.py	Prompts/templates.py and formatting helpers	SQL_WRITER_SYSTEM, SQL_WRITER_USER, SUMMARIZER_SYSTEM, SUMMARIZER_USER, error_section, format_data	Prompt placeholders {schema}, {question}, {error_section}, {sql}, {data}, {error}
src/nl2sql_agent/SQL/text.py	SQL/text.py normalization utilities	strip_sql_fences, truncate	Regex constants _SQL_FENCE_RE, _LEADING_SQL_TOKEN_RE
src/nl2sql_agent/Utils/logging.py	Utils/logging.py logging setup	configure_logging, get_logger	level, json, stdlib interception for httpx/httpcore/...

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
tests/unit/test_agent.py	Workflow behavior contract	Routing/retry/fallback tests for node methods and run/stream	Asserts on retry_count , error , final_answer , node names
tests/unit/test_sql_validation.py	Safety/validation contract	Safety/forbidden/functions tests	Conf/sub query/join/CTE/union allow-list behavior and failure cases

Module 3: Core Execution Flows

This module explains the real operational paths using concrete symbols and data shapes.

3.1 Core data contracts

AgentState shape (src/nl2sql_agent/agent/state.py)

```
# TypedDict, total=False
{
    "question": str,
    "schema": str,
    "sql_query": str,
    "sql_unsafe_reason": str,
    "result": str,
    "raw_rows": list[tuple[object, ...]],
    "columns": list[str],
    "row_count": int,
    "error": str,
    "retry_count": int,
    "max_retries": int,
    "final_answer": str,
}
```

Important note: **AgentState** is declared with **total=False**. That means each node can return partial updates instead of a full state object.

QueryResult shape (src/nl2sql_agent/db/database.py)

```
QueryResult(
    columns: tuple[str, ...],
    rows: tuple[tuple[object, ...], ...],
)
```

```

        row_count: int
    )

```

Sidebar config payload (src/nl2sql_agent/ui/components.py)

```

{
    "provider": Provider,          # "ollama" / "gemini" / "openai" / "anthropic"
    "model": str | None,
    "api_key": str | None,
    "ollama_base_url": str | None,
}

```

Initial workflow input shape (run/stream in src/nl2sql_agent/agent/workflow.py)
Both run() and stream() start with:

```

{
    "question": question,
    "retry_count": 0,
    "max_retries": int(max_retries or self.settings.max_retries),
    "error": "",
}

```

3.2 Flow A: CLI ask request

Entrypoint chain:

1. main() parses args via build_parser().
2. cmd_ask(args) calls _build_agent(...).
3. _build_agent:
 - loads settings = get_settings()
 - applies CLI overrides (provider/model/api_key)
 - builds llm = build_chat_model(settings)
 - returns NL2SQLAgent(llm, settings=settings)
4. cmd_ask calls agent.run(args.question).
5. Prints result["final_answer"]; if --show-sql, also prints result["sql_query"].

Short code fragment from real path:

```

result = agent.run(args.question)
print(result.get("final_answer", ""))
if args.show_sql:
    print("\n--- SQL ---\n", result.get("sql_query", ""), sep="")

```

Input shape: - args.question: str - optional overrides: args.provider, args.model, args.api_key

Output shape: - terminal text; underlying result dict includes final_answer, sql_query, error, columns, raw_rows, row_count.

3.3 Flow B: Streamlit chat request

Entrypoint chain:

1. `streamlit_app.main()` configures page + logging.
2. `cfg = render_sidebar()` returns provider/model/api key/base URL.
3. User input via `st.chat_input()` or `pending_question`.
4. Build model with `build_chat_model(settings=settings, provider=provider, model=model)`.
5. Construct `agent = NL2SQLAgent(llm, settings=settings)`.
6. Execute streamed run: `events = list(agent.stream(user_query))`.
7. Iterate node events; update status UI and accumulate `final_state.update(update)`.
8. Render final output with `render_run_result(...)`.
9. Append assistant message to `st.session_state.messages`.

Short code fragment from real path:

```
events = list(agent.stream(user_query))
for node_name, update in events:
    final_state.update(update)
```

Input shape: - `user_query`: `str` - sidebar config from `render_sidebar()`

Output shape: - UI answer, SQL expander, optional dataframe preview from `columns + raw_rows`.

3.4 Flow C: Agent internals (LangGraph node-by-node)

`NL2SQLAgent.get_workflow()` defines this graph:

1. `fetch_schema(state)`:
 - calls `self.db.get_schema_text()`
 - returns `{"schema": schema}`
2. `write_sql(state)`:
 - reads: `question`, `schema`, `previous error/sql_unsafe_reason`
 - builds `SQL_WRITER_USER` prompt
 - LLM call with `system+user` messages
 - cleans output via `strip_sql_fences()`
 - returns:
 - `sql_query`
 - incremented `retry_count`
 - clears stale `error`, `sql_unsafe_reason`, `result`
3. `check_security(state)`:
 - runs `validate_sql(sql, self.policy)`
 - if failure: returns `error`, `sql_unsafe_reason` (and transient `sql_safe=False`)
 - if success: clears `error`
4. `route_after_security(state)`:
 - if `error` exists -> "summarizer"

- else -> "executor"
- 5. `execute_sql(state)`:
 - calls `self.db.execute(sql)`
 - on success returns markdown `result`, `raw_rows`, `columns`, `row_count`, cleared `error`
 - on `sqlite3.Error` returns error string and empty data containers
- 6. `route_after_execute(state)`:
 - if `error` and `retry_count < max_retries` -> "writer"
 - otherwise -> "summarizer"
- 7. `summarize_result(state)`:
 - builds summarizer prompt from `question`, `sql_query`, formatted `result`, `error`
 - LLM generates `final_answer`
 - fallback path via `_fallback_answer` if LLM fails or returns empty
 - returns `{"final_answer": answer, "result": answer}`

3.5 Flow D: Database initialization + seed behavior

Where it happens:

1. In `NL2SQLAgent.__init__`:
 - `self.db = Database(...)`
 - `self.db.ensure_schema(seed=self.settings.db_seed)`
2. `Database.ensure_schema(seed=True)`:
 - creates parent directories
 - executes DDL for `departments` and `employees`
 - inserts `SEED_DEPARTMENTS` and `SEED_EMPLOYEES` using `INSERT OR IGNORE`
 - guarded by `_init_lock` and `_initialized` flag for idempotency

Tables and seed columns:

1. `departments(dept_id, dept_name, location)`
2. `employees(emp_id, name, salary, dept_id)` with FK `employees.dept_id` -> `departments.dept_id`

3.6 Flow E: Safety gate behavior details

`validate_sql()` in `security/sql_validator.py` enforces:

1. SQL must parse and be non-empty (`parse_sql`).
2. Exactly one statement.
3. Top-level must be `SELECT` or set op (`UNION/INTERSECT/EXCEPT`) if policy allows.
4. Optional policy bans:
 - subqueries
 - joins
 - aggregates

- CTEs
 - union
5. Dangerous functions are blocked (`load_extension`, `readfile`, `writefile`, `edit`, `shell`, `system`).
 6. Additional banned keyword scan blocks destructive operations.

Output contract: - Success: returns list of `sqlglot.expressions.Select`. - Failure: raises `SQLValidationError` with user-facing message.

Why this matters for learners:

1. This safety gate is the key trust boundary in the system.
2. It runs before query execution.
3. It is configurable through `Settings` flags that map to `SQLPolicy`.

Module 4: Setup & Run Guide

This section is static and based on repo files (`README.md`, `pyproject.toml`, `source`).

4.1 Prerequisites

1. Python 3.12.10 (from `pyproject.toml`: `requires-python = ">=3.12.10,<3.13"`).
2. `uv` for environment/dependency management.
3. Optional but default runtime path: local Ollama (`NL2SQL_PROVIDER=ollama`).

4.2 Install on a clean machine

```
git clone <repo-url>
cd natural-language-to-sql-agent
uv venv --python 3.12.10
uv sync --extra dev
uv pip install -e .
```

4.3 Environment configuration (.env or shell env)

Core Settings keys (`src/nl2sql_agent/config/settings.py`) Use these as `NL2SQL_*` variables:

1. `NL2SQL_PROVIDER`
2. `NL2SQL_MODEL`
3. `NL2SQL_OLLAMA_BASE_URL`
4. `NL2SQL_OLLAMA_KEEP_ALIVE`
5. `NL2SQL_DB_PATH`
6. `NL2SQL_DB_SEED`
7. `NL2SQL_DB_MAX_ROWS`
8. `NL2SQL_DB_QUERY_TIMEOUT_SECONDS`
9. `NL2SQL_MAX_RETRIES`

10. NL2SQL_LLM_TEMPERATURE
11. NL2SQL_LLM_MAX_TOKENS
12. NL2SQL_LLM_REQUEST_TIMEOUT_SECONDS
13. NL2SQL_SQL_ALLOW_SUBQUERIES
14. NL2SQL_SQL_ALLOW_JOINS
15. NL2SQL_SQL_ALLOW_AGGREGATES
16. NL2SQL_SQL_ALLOW_CTE
17. NL2SQL_LOG_LEVEL
18. NL2SQL_LOG_JSON

Provider API key env names used by UI helper `env_var_for()` in `settings.py` maps:

1. `openai` -> `OPENAI_API_KEY`
2. `gemini` -> `GOOGLE_API_KEY`
3. `anthropic` -> `ANTHROPIC_API_KEY`

Minimal `.env` examples:

```
# Local-first default path
NL2SQL_PROVIDER=ollama
NL2SQL_MODEL=phi4-mini:3.8b
NL2SQL_OLLAMA_BASE_URL=http://localhost:11434
NL2SQL_DB_PATH=company.db
NL2SQL_DB_SEED=true

# Cloud provider example (OpenAI)
NL2SQL_PROVIDER=openai
NL2SQL_MODEL=gpt-4o-mini
OPENAI_API_KEY=your_key_here
NL2SQL_DB_PATH=company.db
```

4.4 Typical run commands

Start Streamlit UI

```
uv run streamlit run src/nl2sql_agent/ui/streamlit_app.py
```

Ask one question in CLI

```
uv run nl2sql-agent ask "How many employees are in each department?"
uv run nl2sql-agent ask --show-sql "What is the total salary in Engineering?"
```

Show resolved runtime config

```
uv run nl2sql-agent config
```

Serve command wrapper (invokes Streamlit via CLI)

```
uv run nl2sql-agent serve --host 127.0.0.1 --port 8501
```

4.5 Database migration/seeding steps

There is no migration framework (no Alembic/Flyway-style migration scripts in this repo).

Current behavior:

1. Schema creation is runtime and idempotent via `Database.ensure_schema()`.
2. Seeding is controlled by `db_seed` (`NL2SQL_DB_SEED`, default `True`).
3. Backward-compatible helper exists: `setup_db(path, seed=True)`.

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan (zero to hero)

1. Read `README.md` sections: architecture, workflow, configuration, CLI/UI.
2. Read `src/nl2sql_agent/config/settings.py` to understand runtime knobs.
3. Read `src/nl2sql_agent/db/database.py` + `db/seed.py` for actual data model.
4. Read `src/nl2sql_agent/security/sql_validator.py` for guardrail logic.
5. Read `src/nl2sql_agent/prompts/templates.py` for prompt contracts.
6. Read `src/nl2sql_agent/llm/factory.py` for provider abstraction.
7. Read `src/nl2sql_agent/agent/state.py` then `agent/workflow.py` linearly.
8. Read `src/nl2sql_agent/ui/streamlit_app.py` + `ui/components.py`.
9. Read `src/nl2sql_agent/cli.py`.
10. Confirm understanding through tests:
 - `tests/unit/test_agent.py`
 - `tests/unit/test_sql_validator.py`
 - `tests/unit/test_database.py`
 - `tests/unit/test_cli.py`

5.2 Practice exercises

1. Trace one full run: Starting from `cmd_ask`, list each function called until `final_answer` is printed. File focus: `src/nl2sql_agent/cli.py`, `src/nl2sql_agent/agent/workflow.py`.
2. State-contract drill: Write the minimal `AgentState` keys required for `write_sql()` to run without missing context. File focus: `src/nl2sql_agent/agent/state.py`, `src/nl2sql_agent/agent/workflow.py`.

3. Security policy scenario: Explain what happens if `allow_joins=False` and the model generates a JOIN query. File focus: `src/nl2sql_agent/security/sql_validator.py`.
4. Retry logic reasoning: For `retry_count=2`, `max_retries=3`, and execution error present, what node runs next and why? File focus: `src/nl2sql_agent/agent/workflow.py`.
5. Data contract check: What are the exact types for `columns`, `raw_rows`, and `row_count` returned by `execute_sql()`? File focus: `src/nl2sql_agent/db/database.py`, `src/nl2sql_agent/agent/workflow.py`.
6. Config mapping task: Map one settings field to an environment variable and describe where it affects runtime behavior. File focus: `src/nl2sql_agent/config/settings.py`.
7. UI behavior task: Explain when `list_models()` is called in the Streamlit app and where the resulting list is stored. File focus: `src/nl2sql_agent/ui/streamlit_app.py`, `src/nl2sql_agent/ui/components.py`.
8. Extension design: Identify all places to modify when adding a new provider named `xai` (do not implement, just locate files/functions). File focus: `src/nl2sql_agent/config/settings.py`, `src/nl2sql_agent/llm/factory.py`, `src/nl2sql_agent/ui/components.py`.

5.3 Solution outlines

1. Exercise 1 outline: `main` -> `parse_args` -> `cmd_ask` -> `_build_agent` -> `get_settings` -> `build_chat_model` -> `NL2SQLAgent.run` -> `get_workflow/invoke` -> `final dict` -> `print(final_answer)`.
2. Exercise 2 outline: Minimum practical keys are `question`, `schema`; optional prior context is `error` or `sql_unsafe_reason`; `retry_count` defaults to 0 if missing.
3. Exercise 3 outline: `check_security` calls `validate_sql`; validator raises `SQLValidationError("JOIN clauses are not allowed.")`; node returns `error + sql_unsafe_reason`; `route_after_security` sends flow to `summarizer`.
4. Exercise 4 outline: `route_after_execute` checks `err` and `retry < max_retries`; with `2 < 3`, it routes to `"writer"` for another SQL rewrite attempt.
5. Exercise 5 outline: `execute_sql` returns `columns: list[str]`, `raw_rows: list[tuple[object, ...]]`, `row_count: int`.
6. Exercise 6 outline: Example: `Settings.db_max_rows` from `NL2SQL_DB_MAX_ROWS` influences `Database(max_rows=...)`; `execute()` fetches `max_rows + 1` then truncates.

7. Exercise 7 outline: `list_models()` runs when `refresh_trigger_{provider}` is set (button click path in sidebar/main). Results are stored in `st.session_state[f"models_{provider}"]`.
8. Exercise 8 outline: Modify `Provider` literal in `config/settings.py`, add `_build_xai` and `listing/fallback` in `llm/factory.py`, and add provider option/default model handling in `ui/components.py`; optionally adjust `docs/tests`.

Verification Checklist

Use this to self-check mastery:

1. Can you explain the full node order in `NL2SQLAgent` and both routing decisions?
2. Can you describe exactly how unsafe SQL is blocked before execution?
3. Can you enumerate the most important `AgentState` keys and what writes each one?
4. Can you explain how retries are triggered and when they stop?
5. Can you show where schema creation and seed insertion occur?
6. Can you explain how provider/model selection flows from UI/CLI into `build_chat_model()`?
7. Can you point to where final answer fallback behavior is implemented?
8. Can you map at least five environment settings to their runtime effects?